Hapvida: saída do Liferay até 2028

Arquitetura escolhida, resultados da PoC com o HML e plano para 2027

Everton Lupi · Novembro de 2026 · Versão 2 (substitui a proposta inicial com Payload) · Documento para discussão interna
Números da PoC: Liferay de HML, novembro de 2026. Licenças e recursos de fornecedores devem ser revalidados antes da decisão.

1. Resumo

O Liferay deixa de ser usado a partir de 2028. A proposta é substituí-lo por uma stack composta: **Strapi 5** como CMS headless, **Next.js** para o site e os portais, e serviços gerenciados do **Azure** para identidade, arquivos, busca e hospedagem. Os agentes de IA trabalham pelo **MCP**, sempre gerando rascunhos para revisão humana.

Uma PoC rodando localmente já leu o site **Notredame** do Liferay de HML (somente leitura) e reconstruiu o conteúdo nessa stack:

31
páginas migradas como rascunho

86%
dos blocos convertidos automaticamente (139 de 160)

17
content types gerados a partir dos Objects

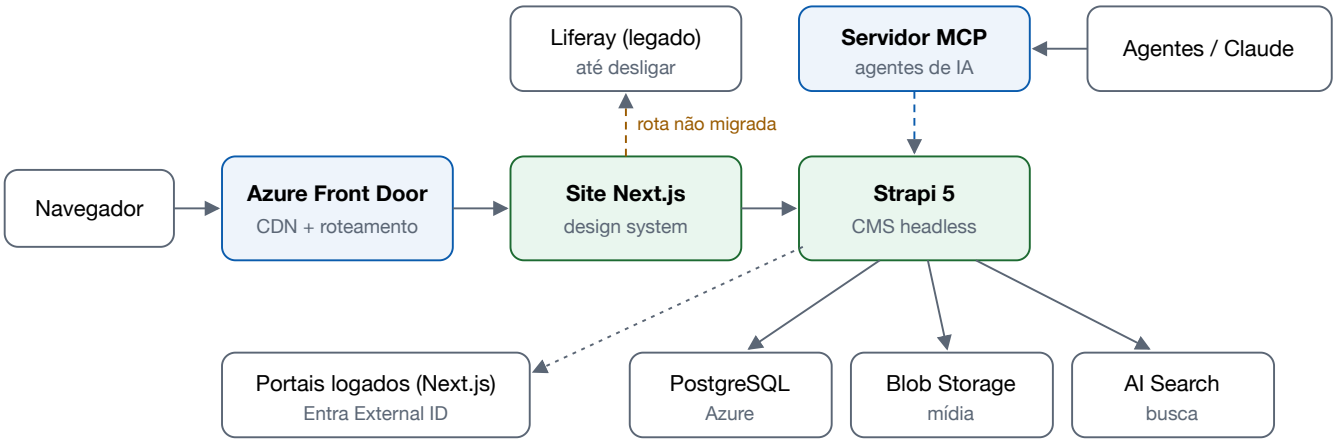
7.4 mil
registros importados, 0 erros (6.801 unidades)

> **Decisões tomadas:** CMS = Strapi 5 (edição Community). Front-end = Next.js com design system próprio. Infraestrutura de HML/produção = Azure. Migração = importadores determinísticos + agentes de IA para revisão e conteúdo livre.

2. Arquitetura alvo (HML e produção no Azure)

Função no Liferay	Na stack nova	**Serviço no Azure** (na PoC local)
Web Content, Structures, Objects	Content types do Strapi (os de Objects são gerados automaticamente)	Strapi em **Azure Container Apps** + **Azure Database for PostgreSQL**
Fragments e Page Builder	Blocos do design system (dynamic zone do Strapi + componentes React)	Site Next.js em **Container Apps**
Master pages	Content type "Modelo de página" (cabeçalho, rodapé e zonas fixas)	A implementar
Themes / Style Books	Tokens CSS por marca, escolhidos no Strapi (Hapvida / NotreDame)	Código do site (design system)
Usuários e SSO dos portais	OIDC padrão nos apps Next.js	**Microsoft Entra External ID** (PoC: Keycloak)
Login de funcionários	SSO corporativo	**Microsoft Entra ID** (no admin do Strapi exige licença Enterprise)
Documents & Media	Media Library do Strapi	**Azure Blob Storage** + **Azure Front Door** (PoC: RustFS)
Busca (Elasticsearch)	Índice atualizado ao publicar	**Azure AI Search** (PoC: Meilisearch)
Forms	Content types Formulário + Envios	Strapi

Roteamento durante a migração	Strangler: rota migrada vai para o site novo, o resto para o Liferay	**Azure Front Door** + regra no Next.js (PoC: nginx)
Integração com agentes	Servidor MCP com ferramentas de domínio	Container Apps; Claude via **Microsoft Foundry** (opcional)
Segredos e logs	—	**Key Vault**, **Azure Monitor / Application Insights**



3. Como as páginas são renderizadas

O Strapi guarda **só dados** (JSON). HTML, CSS e comportamento ficam no código do site, que é o design system. No Liferay é o oposto: cada fragment guarda o próprio HTML, CSS e JS dentro do CMS.

1. O navegador pede uma URL. O site Next.js busca a página no Strapi pela API REST e recebe a lista de blocos, por exemplo `banner-topo`, `grade-cards`, `lista-colecao`.
2. Cada bloco é trocado pelo seu componente React. Blocos de coleção (FAQ, notícias, planos) fazem uma segunda busca no Strapi.
3. O cabeçalho e o rodapé vêm da Navegação (no futuro, do Modelo de página). A marca escolhida no Strapi vira o atributo `data-brand` da página.
4. Tudo é montado **no servidor**: o navegador recebe HTML pronto (bom para SEO e celulares) e um único CSS pequeno. As imagens vêm direto do Blob Storage/CDN.

CSS	Onde fica e como muda
Cores de cada marca	Tokens em um arquivo do design system. Trocar a marca no Strapi troca as cores do site e dos portais, sem deploy.
Visual de cada bloco	No componente do bloco. Mudou o componente, mudam todas as páginas que o usam.
O que o editor controla	Conteúdo e opções previstas (ex.: estilo do card "padrão / branco / arredondado"). O editor não escreve CSS, o que evita duplicações como `banner-estatico` e `banner-estático` no Liferay atual.

Rascunhos: o botão "Abrir preview" do Strapi mostra a página no site em modo rascunho antes de publicar. **Em produção** as páginas ficam em cache na CDN, e o Strapi avisa o site para atualizar no momento da publicação.

Master pages: o Strapi não tem o recurso nativo. O equivalente é um content type "Modelo de página", com cabeçalho, rodapé e zonas de blocos antes e depois do conteúdo, ligado a cada página. Uma mudança no modelo vale para todas as páginas que o usam.

4. O que a PoC comprovou

Migração do site Notredame (HML, somente leitura)

Item	Resultado
Estrutura do site	0 Web Contents. O site é feito de **31 páginas de fragments** e **21 Objects**, com 185 documentos.
Objects → content types	17 Objects de conteúdo viraram content types **automaticamente**, com campos, listas e relações. 4 Objects operacionais (fila, sincronização, LGPD) ficaram de fora de propósito.
Registros	6.801 unidades, 313 especialidades, FAQ, notícias, blog, banners, planos e outros. **0 erros** e 6.174 relações religadas. A importação pode ser repetida sem duplicar.
Páginas	31 páginas como rascunho, com SEO, imagens no armazenamento novo e links internos reescritos. **139 de 160 blocos (86%)** convertidos automaticamente.
Listas de conteúdo	Asset Lists e Blueprints viram blocos de lista com os mesmos itens e a mesma ordem do Liferay.
Ainda por reconstruir	21 blocos: fragments `imagem-com-lista`, `tabs`, `suporte`; widgets React de filtro e busca; formulário de Ouvidoria (portlet Java); 5 páginas definidas.

Capacidades demonstradas

- **Troca gradual (strangler):** rota não migrada continua no Liferay. Ao publicar a página nova, o redirect da URL antiga é criado sozinho (308) e a rota passa para a stack nova.
- **Busca:** conteúdo publicado entra no índice automaticamente. Rascunho não entra.
- **Tema por marca:** Hapvida ↔ NotreDame em um clique, refletindo no site e no portal.
- **Portal logado:** SSO via OIDC, carteirinha (dados simulados das APIs core) e avisos vindos do CMS.
- **Formulários** montados no CMS, com os envios salvos.
- **Permissões por token:** o site só lê; o MCP cria e edita, mas não apaga; a migração tem acesso total.

5. Agentes de IA

> **Governança:** agentes só criam e editam **rascunhos**. A publicação é sempre de uma pessoa no admin do Strapi. Em saúde isso é obrigatório (ANS, CDC, LGPD).

Uso	Como funciona
Inventário do Liferay	Relatório só leitura de páginas, fragments, Objects, coleções e widgets. Já rodado no Notredame; base para estimar os outros sites.
Migração de conteúdo livre	O agente lê o conteúdo legado, reescreve em seções limpas e grava rascunho pelo MCP. Pode usar o Claude, inclusive via Microsoft Foundry no Azure.
Revisão de compliance	Alertas automáticos para revisão humana: promessas absolutas, carência, preços e datas antigas, arquivos legados.
Agentes das equipes	Qualquer cliente MCP (ex.: Claude Code) lista páginas, cria rascunhos de páginas e notícias e consulta o tema.

A migração de **dados estruturados** (Objects e fragments) é feita por importadores determinísticos, não por IA: o resultado é previsível, auditável e repetível.

6. Plano para 2027

Fase	Período	Entregas	Status
0. Inventário	Nov/26 a Jan/27	Inventário de todos os sites (Notredame feito). Classificar cada item em migrar, reescrever ou aposentar.	em andamento
1. Fundação	Fev a Abr/27	Ambiente Azure (HML), design system compartilhado, Modelos de página, blocos restantes, Entra External ID nos portais, AI Search.	a iniciar
2. Troca gradual	Abr a Set/27	Migração em ondas por site com os importadores; publicação após revisão; rotas passam para a stack nova.	a iniciar
3. Portais logados	Paralelo	Reescrever em Next.js (beneficiário, corretor, prestador). É a parte de maior risco.	a iniciar
4. Desligamento	Out a Dez/27	Congelar o Liferay, conferir redirects, desligar.	a iniciar

7. Decisões em aberto e riscos

Tema	Ponto
Licença do Strapi	A PoC usa só a edição gratuita. **SSO no admin (Entra ID), Review Workflows (etapas de aprovação) e Audit Logs são pagos** (Enterprise); Content History e Releases exigem plano pago. Decidir se esses recursos são necessários e confirmar preços com a Strapi.
Troca gradual no Azure	O Front Door não reencaminha para outra origem quando recebe 404. Proposta: o site Next.js decide (rota migrada ou proxy para o Liferay) e o Front Door envia tudo para o site.
Rede credenciada	A base de unidades no HML cresceu de 852 para 6.801 em uma semana. Avaliar um serviço próprio (API + AI Search) em vez de conteúdo editorial no CMS.
Componentes customizados	Widgets React e o portlet da Ouvidoria precisam de reescrita. Os widgets React provavelmente podem ser reaproveitados no Next.js.
Arquivos SVG	O Strapi bloqueia SVG por segurança (4 ícones não migraram). Converter para PNG ou liberar com sanitização.
LGPD e IA	Conteúdo enviado à IA deve excluir dados pessoais. Objects operacionais (ex.: pedidos de exclusão) não são migrados como conteúdo.
Outros sites	Só o Notredame foi inventariado. Rodar o inventário nos demais sites para dimensionar 2027.

8. Próximos passos

1. Rodar o inventário nos demais sites do Liferay de HML.
2. Decidir a licença do Strapi (Community ou Enterprise).
3. Provisionar o ambiente de HML no Azure (Container Apps, PostgreSQL, Blob, Front Door, AI Search, Entra External ID, Key Vault).
4. Construir os blocos restantes e os Modelos de página no design system.
5. Apresentar a PoC às equipes de conteúdo para validar a experiência de edição no Strapi.